DevOps Internship Assignment

Final Technical Report

AI Query System — Cloud Deployment, Automation & Monitoring

Submitted By

Mohammed Asim Omran

Technology Stack

Java 21 | Spring Boot 3 | AWS | GitHub Actions | Amazon CloudWatch | k6

Table of Contents

1. Introduction

The AI Query System is a web-based application built on Spring Boot 3 that integrates with the OpenRouter API to deliver AI-generated responses through a simple, browser-accessible interface. Beyond its function as an intelligent query tool, the project was designed as a hands-on demonstration of the complete DevOps lifecycle — spanning application development, continuous integration and deployment, infrastructure provisioning, monitoring, security hardening, and performance validation.

This report documents the engineering approach taken to design, deploy, and operate the application on Amazon Web Services (AWS). The application is deployed on an Amazon EC2 instance, with deployments automated end-to-end through GitHub Actions. Runtime health and resource utilization are tracked using Amazon CloudWatch, and the system's performance under concurrent load was verified using the k6 load-testing framework.

The purpose of this assignment was not only to build a functioning application, but to develop practical, production-oriented DevOps competencies: infrastructure operations, CI/CD pipeline design, observability, security best practices, and performance benchmarking — all applied to a real, working system rather than a theoretical exercise.

> **Project Snapshot**
> A single Spring Boot application, deployed on AWS EC2, with a fully automated GitHub Actions deployment pipeline, CloudWatch-based monitoring, and k6-validated performance under simulated concurrent load.

2. Project Objectives

The assignment was structured around six core objectives, each targeting a distinct stage of the DevOps lifecycle. Together, they represent a complete path from code to a monitored, secured, and performance-validated production deployment:

- Develop a functional Spring Boot application capable of serving AI-generated responses through a web interface.
- Deploy the application on an Amazon EC2 instance within the AWS cloud environment.
- Configure a continuous integration and continuous deployment (CI/CD) pipeline using GitHub Actions.
- Establish infrastructure and application monitoring using Amazon CloudWatch.
- Validate application performance and reliability under concurrent load using the k6 testing tool.
- Apply cloud security best practices across identity management, secrets handling, and network access control.

Each objective is addressed in a dedicated section of this report, with supporting evidence, diagrams, and configuration details.

3. Technology Stack

The following table summarizes the complete technology stack used across development, deployment, monitoring, and testing for the AI Query System.

Category	Technology
Backend Framework	Spring Boot 3
Programming Language	Java 21
Frontend	HTML, CSS, JavaScript
Build Tool	Apache Maven
Cloud Provider	Amazon Web Services (AWS)
Compute	Amazon EC2
Storage	Amazon S3
Monitoring & Observability	Amazon CloudWatch
CI/CD Automation	GitHub Actions
Performance Testing	k6
Version Control	Git & GitHub
AI Provider	OpenRouter API

This stack was deliberately kept lean: every component listed above is actively used in the deployed system, avoiding unnecessary complexity while still covering the full breadth of a modern DevOps toolchain — from source control through to cloud-native monitoring.

4. System Architecture

The system follows a linear, automation-driven architecture in which a code change made by the developer flows through version control, continuous integration, and remote deployment, ultimately resulting in an updated, running application instance. Figure 1 illustrates the complete end-to-end flow, from the developer's local environment to the live Spring Boot application and its integration with the OpenRouter API.

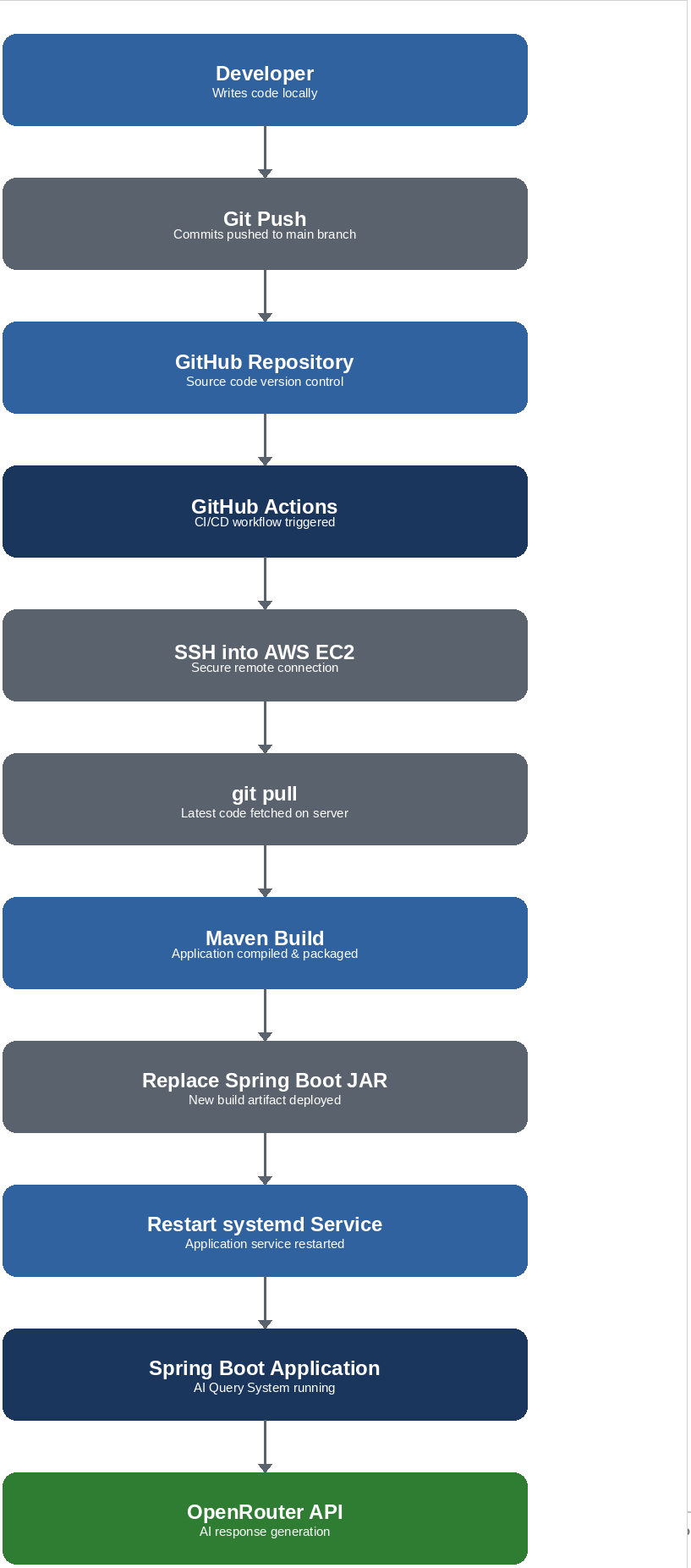
Developer
Writes code locally
Git Push
Commits pushed to main branch
GitHub Repository
Source code version control
GitHub Actions
CI/CD workflow triggered
SSH into AWS EC2
Secure remote connection
git pull
Latest code fetched on server
Maven Build
Application compiled & packaged
Replace Spring Boot JAR
New build artifact deployed
Restart systemd Service
Application service restarted
Spring Boot Application
AI Query System running
OpenRouter API
AI response generation

Figure 1: End-to-end system architecture and deployment flow.

At a high level, the architecture can be described in three logical layers:

Source Control & Automation Layer

The developer commits code changes locally and pushes them to the GitHub repository. This push event acts as the trigger for the automated pipeline, ensuring that no deployment occurs without a corresponding, version-controlled code change.

Deployment Layer

GitHub Actions orchestrates the deployment by establishing a secure SSH session with the target EC2 instance. Once connected, it pulls the latest code, rebuilds the application using Maven, replaces the existing Spring Boot JAR file, and restarts the application via its systemd service definition.

Application Layer

The running Spring Boot application serves the web interface and forwards user queries to the OpenRouter API, which returns AI-generated responses. This layer is the only component directly exposed to end users, while all preceding layers operate as backend automation.

> **Design Observation**
> Keeping the deployment path linear and fully scripted — rather than manual — eliminates configuration drift between environments and ensures every deployment is reproducible and auditable via the GitHub Actions run history.

5. AWS Infrastructure

The AWS infrastructure supporting the AI Query System is intentionally minimal, favoring a small set of well-understood managed services over unnecessary architectural complexity. Figure 2 shows the relationship between the core AWS components used in this project.

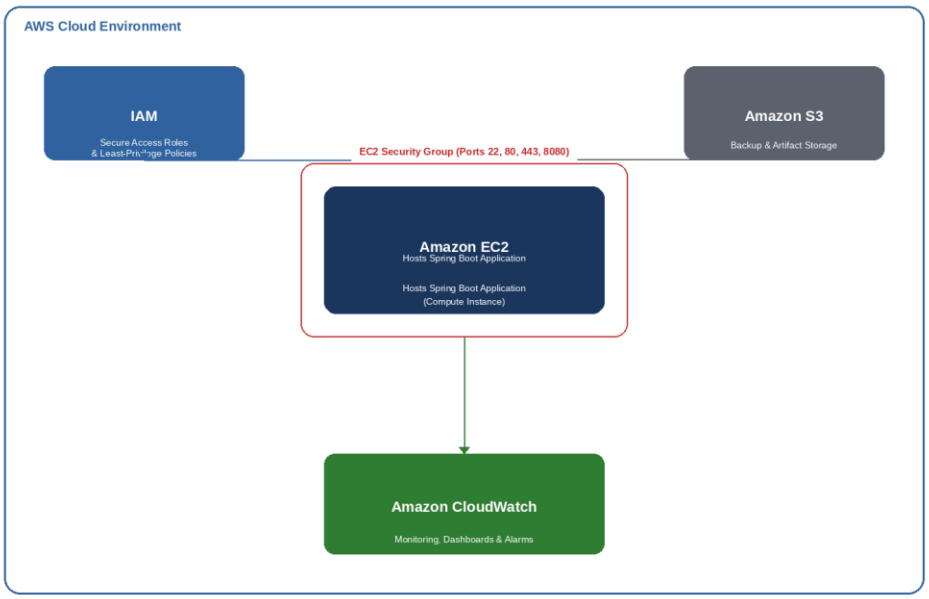

Figure 2: AWS infrastructure and service relationships.

Amazon EC2

Amazon EC2 hosts the Spring Boot application as a long-running process managed by systemd. The instance is the single compute resource in the architecture and is the target of every automated deployment executed by GitHub Actions.

AWS Identity and Access Management (IAM)

IAM provides scoped, role-based access to AWS resources, ensuring that only authorized identities and services can interact with EC2, S3, and CloudWatch. Access is granted following the principle of least privilege, discussed further in the Security section of this report.

Amazon S3

Amazon S3 is configured to support future backup and artifact storage needs — for example, archiving build artifacts or application logs. While not yet central to the live request path, it establishes the storage foundation required for future scaling of the system.

Amazon CloudWatch

CloudWatch continuously monitors both the EC2 instance and the application layer, providing visibility into resource utilization and enabling proactive alerting, as detailed in the Monitoring section.

6. Deployment Process

Deployment of the AI Query System follows a clearly defined, repeatable eight-step process. This process is fully automated after the initial push to the main branch, requiring no manual intervention on the EC2 instance.

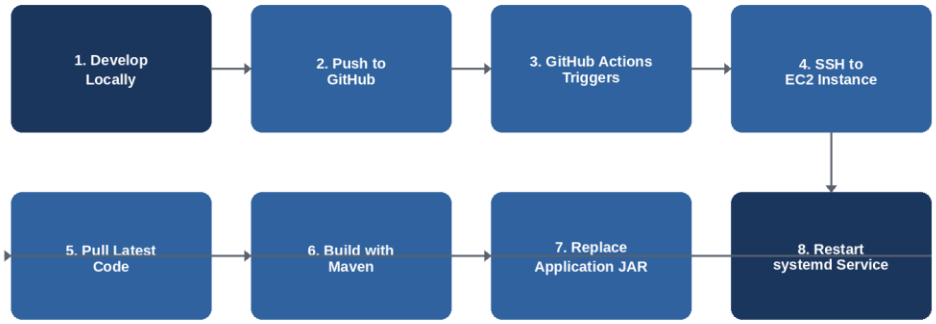

Figure 3: Deployment workflow, from local development to service restart.

Step	Action	Description
1	Develop Locally	Application code is written and tested on the developer's local machine.
2	Push to GitHub	Committed changes are pushed to the main branch of the GitHub repository.
3	GitHub Actions Triggers	The push event automatically triggers the configured CI/CD workflow.
4	SSH to EC2	GitHub Actions establishes a secure SSH connection to the target EC2 instance.
5	Pull Latest Code	The latest source code is pulled onto the EC2 instance via git pull.
6	Build with Maven	The application is compiled and packaged into an executable JAR using Maven.
7	Replace Application JAR	The previous JAR file is replaced with the newly built artifact.
8	Restart systemd Service	The systemd service is restarted, bringing the updated application online.

Important Observation

Because the entire deployment sequence is scripted within GitHub Actions, deployments are consistent regardless of who initiates the push, significantly reducing the risk of human error during release.

7. CI/CD Pipeline

Continuous Integration and Continuous Deployment (CI/CD) is implemented using GitHub Actions, providing a fully automated release path from source code to a running production instance. Every push to the main branch initiates the pipeline shown in Figure 4.

Figure 4: CI/CD pipeline stages executed by GitHub Actions on every push to main.

The pipeline performs the following actions in sequence: it detects the code push, executes the GitHub Actions workflow, opens a secure SSH connection to the EC2 instance, builds the application with Maven and deploys the resulting JAR, and finally restarts the Spring Boot service so that the new version becomes active.

This design ensures that the main branch always reflects what is actually running in production, and that every deployment is traceable to a specific commit and a specific workflow run within GitHub Actions.

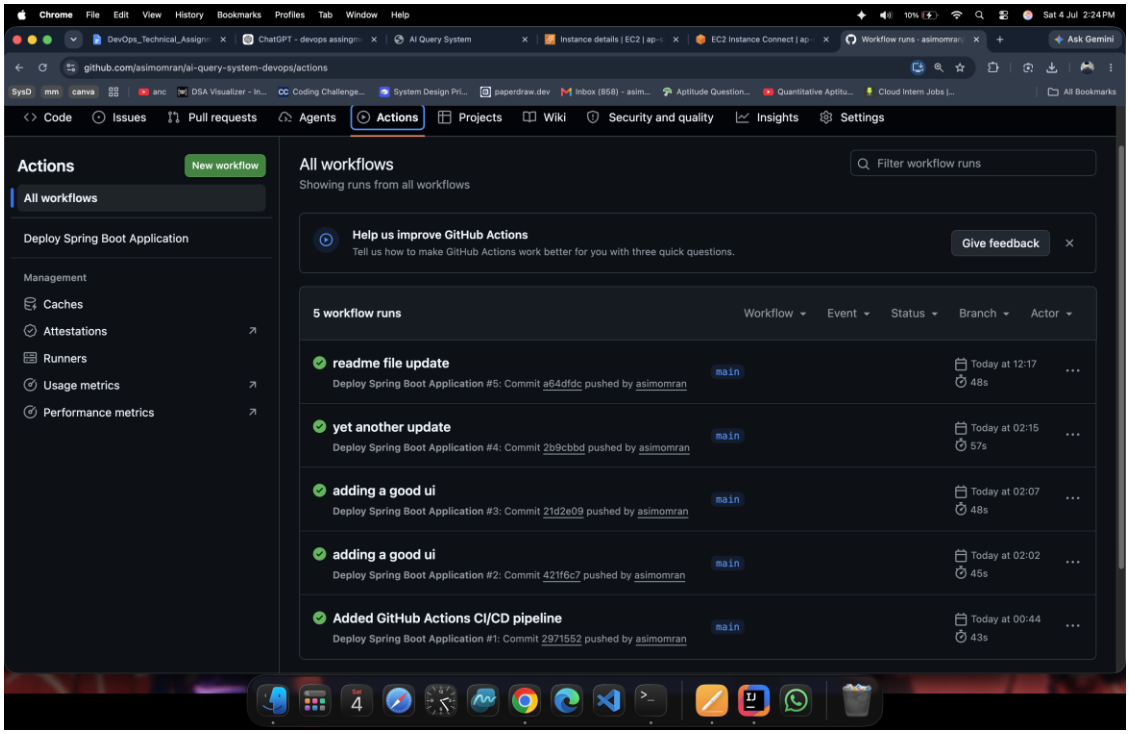

Figure 5: GitHub Actions workflow run — build and deployment log.

8. Monitoring

Effective monitoring is essential to operating any production system reliably, and this project uses Amazon CloudWatch as its primary observability tool. CloudWatch was configured to track CPU Utilization, Network In, and Network Out metrics for the EC2 instance, alongside a CPU-based alarm that provides early warning of resource strain.

Why Monitoring Matters

Monitoring transforms infrastructure from a black box into a system that can be reasoned about. Without visibility into resource usage, issues such as CPU exhaustion, memory pressure, or network saturation would only become apparent once they had already degraded the user experience. By continuously tracking key metrics, CloudWatch enables the identification of performance trends, early detection of abnormal behavior, and a factual basis for capacity planning as the application scales.

Dashboards

The CloudWatch dashboard consolidates the instance's key health indicators into a single view, allowing CPU load, inbound traffic, and outbound traffic to be assessed at a glance. This is particularly valuable immediately after a deployment, when confirming that the new release is behaving normally is a priority.

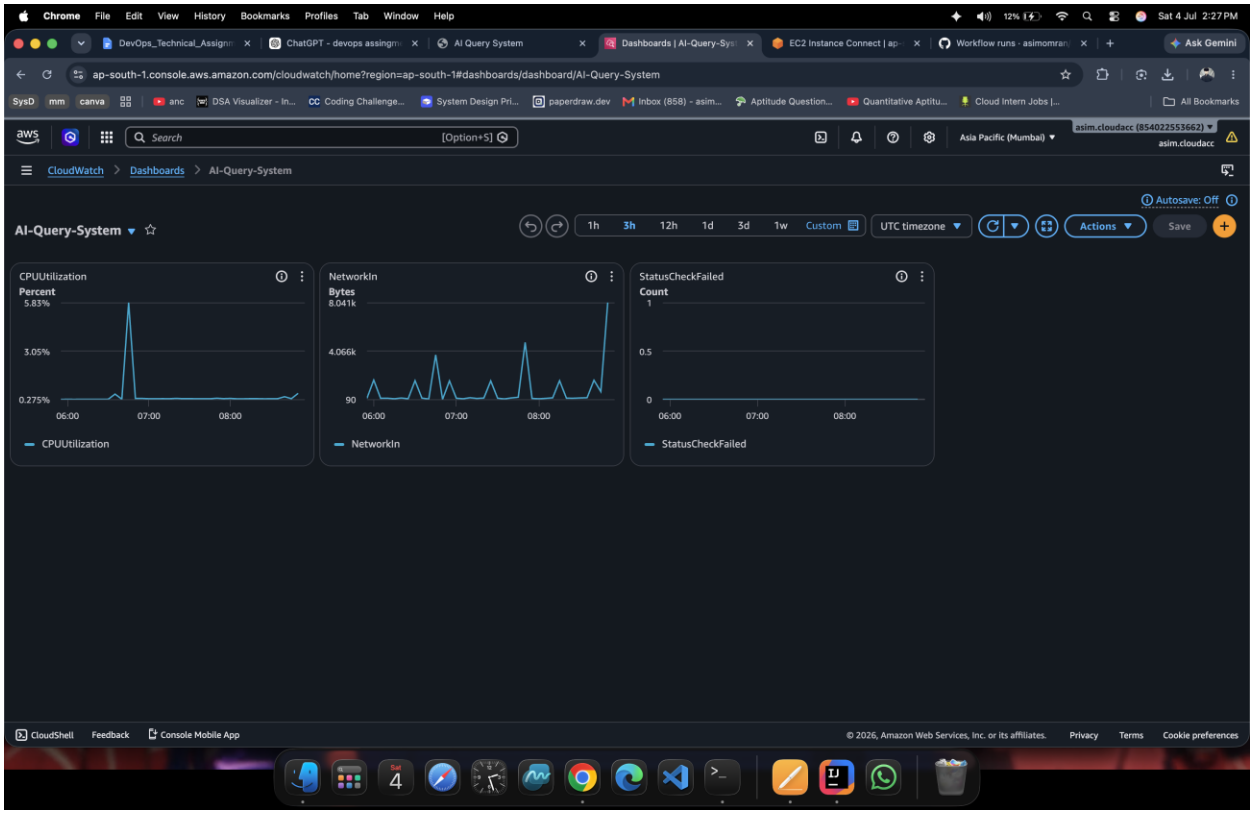

Figure 6: CloudWatch dashboard showing CPU and network metrics.

Alarms

A CloudWatch alarm was configured against the CPU Utilization metric to automatically flag sustained high load. Alarms convert passive metric collection into an active alerting mechanism, ensuring that resource issues are surfaced immediately rather than discovered only during manual inspection.

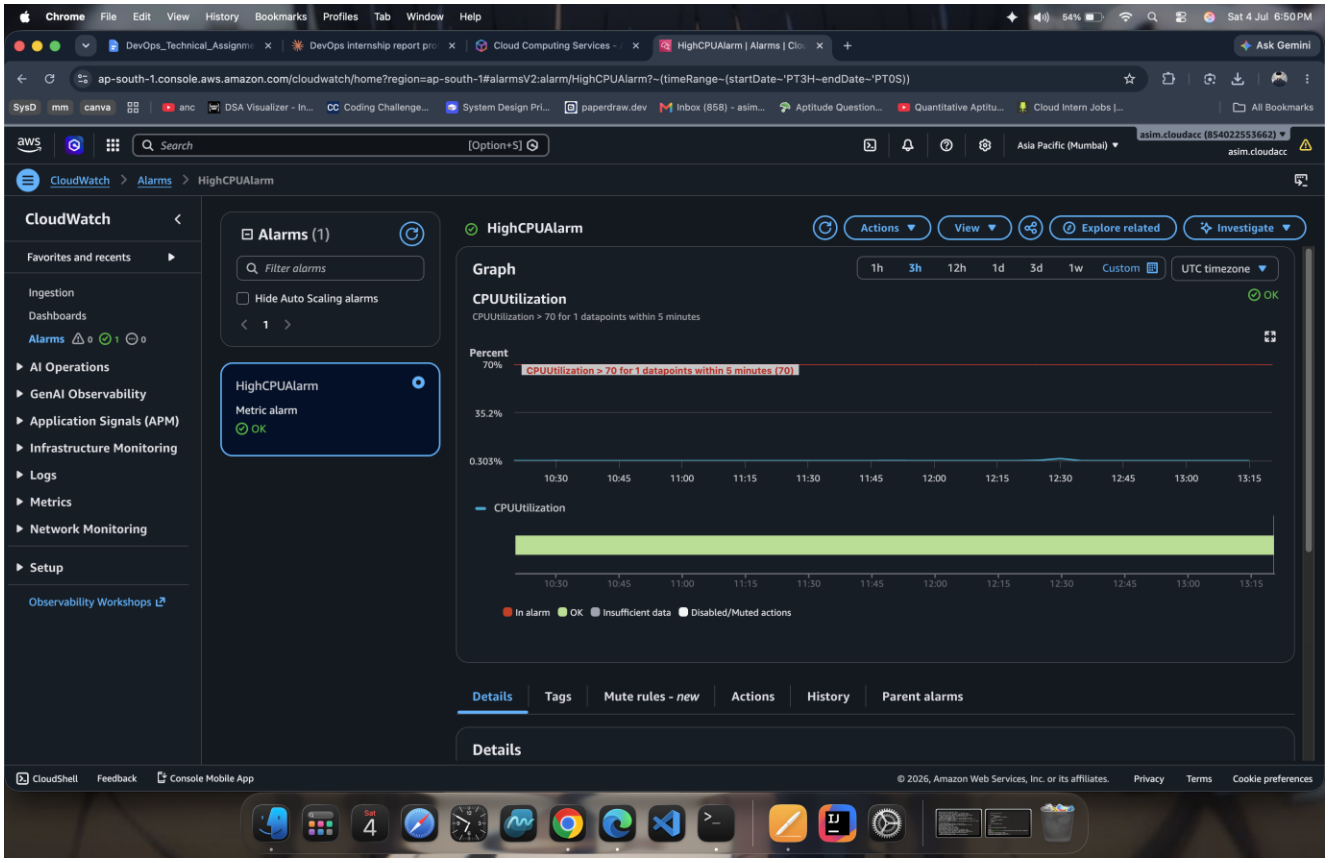

Figure 7: CloudWatch alarm configuration for CPU Utilization.

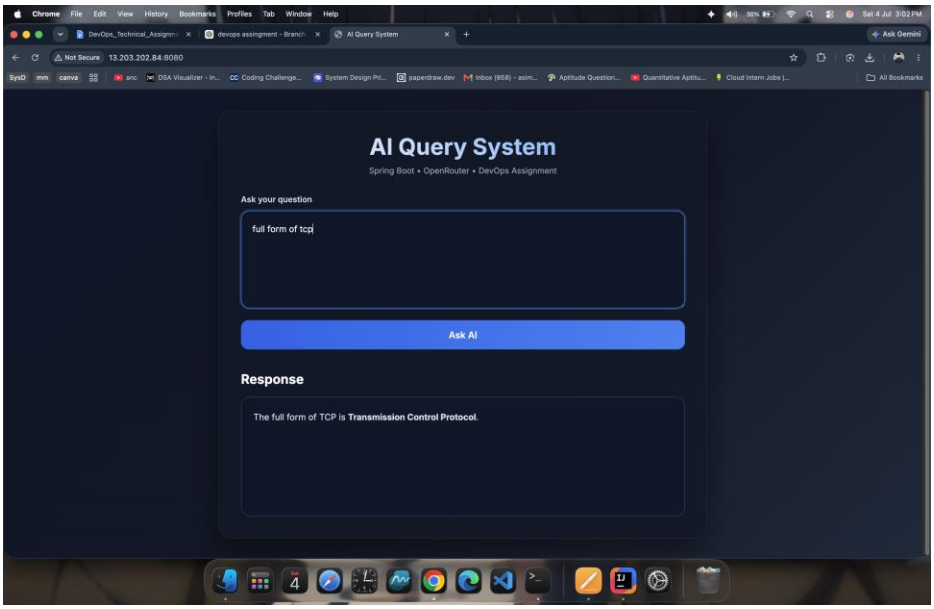

Figure 8: AI Query System web interface.

9. Load Testing

Performance validation was carried out using k6, an open-source load-testing tool well suited for scripting and automating HTTP-based performance tests. The objective was to confirm that the deployed application could handle concurrent traffic without errors and with acceptable response times.

Test Configuration

Parameter	Value
Virtual Users	10
Test Duration	30 seconds

Test Results

Metric	Result
Total Requests	300
Failed Requests	0
Average Response Time	29.39 ms
95th Percentile Response Time	39 ms
Maximum Response Time	44.7 ms

Metric	Result
Throughput	9.7 requests/second

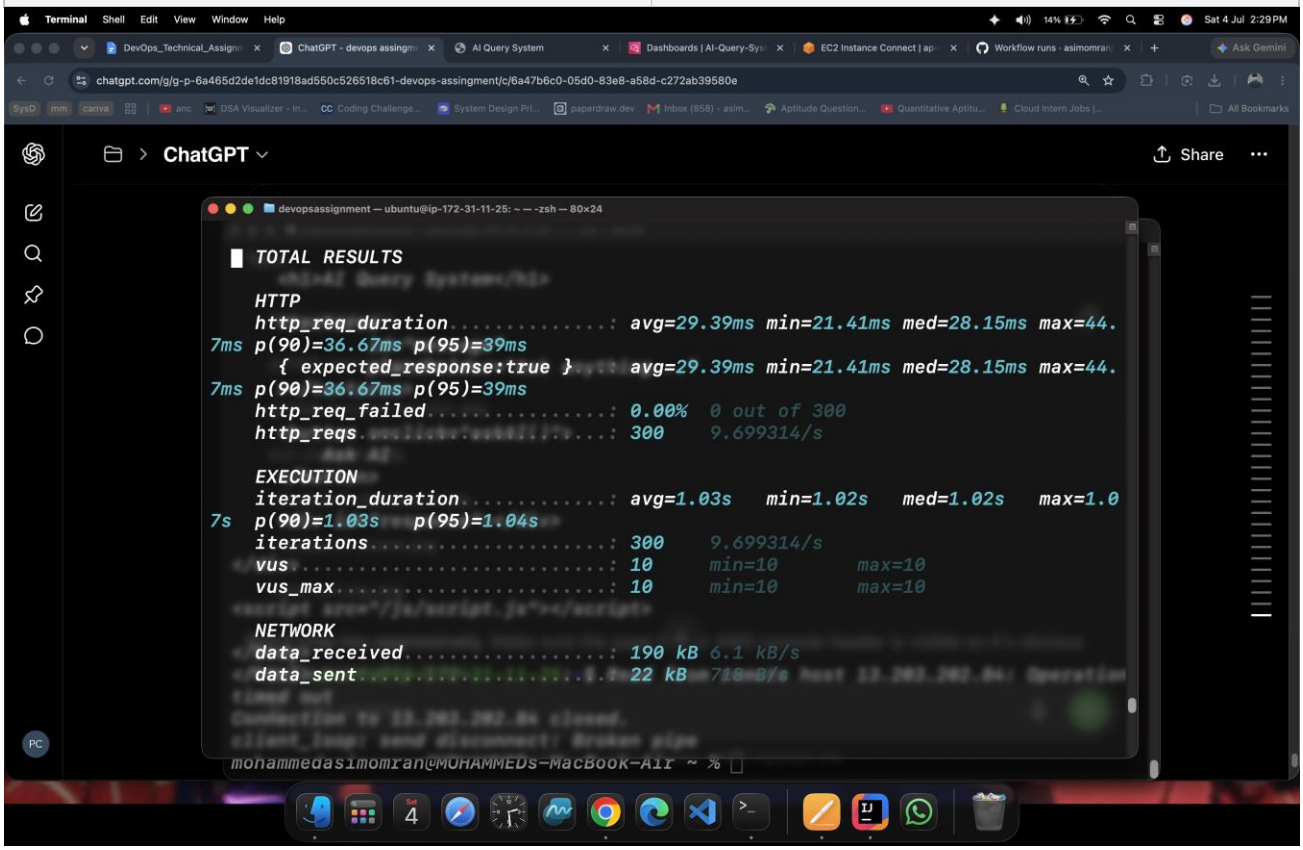

Figure 9: k6 load test execution output.

Observations

The application processed all 300 requests successfully, with zero failures recorded across the test duration — indicating that the deployment is stable under the tested concurrency level. Response times remained low and tightly distributed: the average response time of 29.39 ms and 95th percentile of 39 ms are close to one another, suggesting consistent performance rather than occasional latency spikes. The maximum observed response time of 44.7 ms remained well within an acceptable range for an interactive web application.

A throughput of 9.7 requests per second, sustained without any failed requests, demonstrates that the current single-instance EC2 deployment can comfortably support the tested load of 10 concurrent virtual users. This result also establishes a performance baseline that future load tests — for example, at higher concurrency — can be compared against.

> **Key Result**
> 0 failed requests out of 300, with a 95th percentile response time of just 39 ms — confirming stable performance under the tested load and establishing a reliable baseline for future scalability testing.

10. Security

Security was addressed at multiple layers of the system — identity and access management, secrets handling, network access, and data-at-rest protection. The following subsections describe each control implemented in this project.

IAM Least Privilege

AWS Identity and Access Management (IAM) roles and policies were scoped according to the principle of least privilege, meaning that each identity or service is granted only the specific permissions required to perform its function, and nothing more. This reduces the blast radius of any single compromised credential and limits the actions that could be taken if a role were misused, whether accidentally or maliciously.

GitHub Secrets

Sensitive values required by the CI/CD pipeline — such as SSH credentials and API keys — are stored as encrypted GitHub Secrets rather than being hard-coded into the workflow files or committed to the repository. GitHub injects these secrets into the workflow environment only at runtime, ensuring they are never exposed in logs or visible in the source code.

SSH Authentication

Access to the EC2 instance during deployment is authenticated using SSH key pairs rather than passwords. This provides a significantly stronger authentication mechanism, as it requires possession of a private key rather than knowledge of a guessable secret, and it aligns with AWS's own recommended practice for EC2 instance access.

EC2 Security Groups

Security Groups function as a virtual firewall for the EC2 instance, restricting inbound and outbound traffic to only the ports and protocols required for the application to function — such as SSH for deployment access and HTTP for serving the web application. All other inbound traffic is denied by default, minimizing the instance's exposed attack surface.

Amazon S3 Security

The S3 bucket configured for backups and artifact storage is protected through access policies that restrict who can read from or write to it, along with encryption to protect data at rest. This ensures that stored artifacts remain confidential and are only accessible to authorized IAM identities.

> **Security Principle**
> Every credential in this system — SSH keys, API tokens, and IAM permissions — follows the principle of least privilege: granted narrowly, stored securely, and never exposed in logs, source code, or version control history.

11. Challenges Faced

Building and deploying the AI Query System surfaced a number of practical engineering challenges, spanning application configuration, CI/CD pipeline behavior, and infrastructure connectivity. Each challenge below is documented as a short case study covering the problem encountered, its root cause, the solution applied, and the resulting outcome.

1. OpenRouter API Configuration

Problem: Initial requests to the OpenRouter API failed to return valid responses, preventing the application from generating AI output.

Root Cause: The API key and endpoint configuration were not being loaded correctly into the Spring Boot application's runtime environment.

Solution Implemented: The OpenRouter API key and related configuration values were externalized into Spring Boot's application properties and verified against OpenRouter's expected request format before redeployment.

Outcome: AI query responses were returned successfully and consistently, confirming correct integration with the OpenRouter API.

2. Spring Boot Property Injection

Problem: Certain configuration values were not being picked up by the application at startup, resulting in default or missing values being used instead of the intended configuration.

Root Cause: Property names and environment variable bindings did not match the naming conventions expected by Spring Boot's property injection mechanism.

Solution Implemented: Property keys were audited and corrected to align with Spring Boot's configuration binding rules, and startup logs were reviewed to confirm each value was being injected as expected.

Outcome: All required configuration values were correctly injected at application startup, eliminating the reliance on default fallback values.

3. GitHub Personal Access Token (PAT) Permissions

Problem: The GitHub Actions workflow was unable to complete certain repository or deployment operations due to insufficient permissions on the Personal Access Token in use.

Root Cause: The PAT had been generated with a narrower permission scope than the workflow's operations required.

Solution Implemented: A new PAT was generated with the correct scope for the required repository and deployment operations, and it was stored securely as a GitHub Secret.

Outcome: The GitHub Actions workflow completed all repository operations without permission errors on subsequent runs.

4. GitHub Actions SSH Timeout

Problem: The SSH step within the GitHub Actions workflow intermittently timed out before completing the connection to the EC2 instance.

Root Cause: Default SSH connection timeout values in the workflow were too short relative to the EC2 instance's response time under certain network conditions.

Solution Implemented: SSH timeout and retry parameters within the workflow were adjusted, and connectivity to the EC2 instance's security group rules was re-verified to ensure the required port was reliably reachable.

Outcome: SSH connections from GitHub Actions to the EC2 instance completed reliably, removing the intermittent deployment failures.

5. EC2 Deployment Troubleshooting

Problem: After certain deployments, the application failed to restart correctly, leaving the previous version running or the service in a failed state.

Root Cause: The systemd service definition and file paths referenced during the JAR replacement step did not consistently match the actual deployment directory structure on the EC2 instance.

Solution Implemented: The systemd service file and deployment script paths were reviewed and corrected, and the restart sequence was tested manually before being re-integrated into the automated pipeline.

Outcome: The application reliably restarted with the newly deployed JAR after every subsequent pipeline run.

6. Load Testing Connectivity

Problem: Initial k6 load test runs failed to connect to the deployed application, returning connection errors instead of response metrics.

Root Cause: The load testing tool was targeting an incorrect endpoint or port, and EC2 Security Group rules had not yet been configured to permit the required inbound traffic for testing.

Solution Implemented: The k6 test script's target URL was corrected, and the EC2 Security Group was updated to allow the necessary inbound access for load testing.

Outcome: k6 successfully executed the full load test against the live application, producing the results documented in the Load Testing section of this report.

12. Future Improvements

While the current implementation successfully satisfies the assignment's objectives, several enhancements have been identified that would further strengthen the system's scalability, security, and operational maturity. These are presented as forward-looking recommendations rather than implemented features.

Containerization with Docker

Packaging the Spring Boot application as a Docker container would decouple it from the host EC2 instance's specific environment configuration, improving portability and making local development environments more consistent with production.

Nginx with HTTPS

Introducing Nginx as a reverse proxy in front of the application, combined with an HTTPS certificate, would encrypt traffic between clients and the server and allow for cleaner request routing, request buffering, and future support for multiple backend instances.

Infrastructure as Code with Terraform

Defining the AWS infrastructure — EC2 instance, Security Groups, IAM roles, and S3 configuration — using Terraform would make the environment reproducible, version-controlled, and far easier to recreate or modify safely compared to manual console configuration.

Auto Scaling

Configuring an Auto Scaling Group would allow the application to automatically add or remove EC2 instances in response to real traffic demand, improving resilience and cost efficiency compared to a single, statically sized instance.

Blue-Green Deployment

Adopting a blue-green deployment strategy would allow new versions of the application to be deployed to a separate environment and validated before traffic is switched over, reducing the risk of downtime or user-facing errors during releases.

Custom Domain

Mapping a custom domain name to the application, in place of the default EC2 public address, would improve the professionalism and usability of the system and is a prerequisite for a properly configured HTTPS certificate.

> **Roadmap Note**
> These improvements are intentionally sequenced: containerization and Infrastructure as Code establish the foundation, while Nginx, HTTPS, Auto Scaling, and blue-green deployment build on that foundation to improve resilience and user-facing quality.

13. Conclusion

This project successfully demonstrates a complete, functioning DevOps workflow — from application development through automated deployment, infrastructure monitoring, security hardening, and performance validation. The AI Query System was built on Spring Boot 3 and Java 21, deployed on Amazon EC2, and integrated with the OpenRouter API to deliver AI-generated responses through a web interface.

A fully automated CI/CD pipeline, implemented using GitHub Actions, ensures that every change pushed to the main branch is built, deployed, and brought online without manual intervention. Amazon CloudWatch provides continuous visibility into system health through dashboards and alarms, while load testing with k6 confirmed that the deployed application handles concurrent traffic reliably, with zero failed requests and consistently low response times across 300 test requests.

Security was addressed holistically, applying least-privilege IAM policies, encrypted secrets management, key-based SSH authentication, restrictive network access controls, and secured storage. The engineering challenges encountered during the project — spanning API configuration, property injection, access permissions, and deployment troubleshooting — were each resolved methodically and are documented in this report as practical case studies.

Taken together, this assignment reflects not only the successful delivery of a working cloud-deployed application, but the development of hands-on, production-relevant DevOps skills: infrastructure operations, deployment automation, observability, security engineering, and performance benchmarking. The future improvements outlined in this report provide a clear and realistic path toward a more scalable, resilient, and production-grade deployment.

— End of Report —